

robocar-unified

Single-Board AI Robot Car

Assembly, Wiring & Firmware Build Guide

A hands-on guide to assembling the consolidated robocar on a **Seeed Studio XIAO ESP32-S3 Sense**. Camera capture, Gemini AI planning, motor control, and peripherals all run on one module.

Firmware version	v0.1.2
Target MCU	ESP32-S3 (8 MB PSRAM / flash)
Toolchain	ESP-IDF v5.4 (containerized)
Difficulty	Intermediate · ~2-3 h

Pin assignments in this guide mirror `main/pin_config.h`, which is authoritative.
All components must share a common ground.

Contents

1 · Overview	3
2 · Bill of Materials	3
Tools required	3
3 · System Architecture	4
4 · Wiring Reference	4
4.1 · XIAO ESP32-S3 GPIO assignments	4
4.2 · I ² C topology (TCA9548A @ 0x70)	5
4.3 · PCA9685 channel map (0x40, 200 Hz)	5
4.4 · Ultrasonic rangefinder	5
5 · Power	5
6 · Assembly Steps	6
7 · Build & Flash the Firmware	6
8 · First Boot & Provisioning	7
8.1 · WiFi over Bluetooth (Improv)	7
8.2 · Discovery & AI backend	7
8.3 · Over-the-air updates	7
9 · Functional Checkout	7
10 · Troubleshooting	7

1 · Overview

The **robocar-unified** project consolidates the original dual-ESP32 robocar (a Heltec main controller plus an ESP32-CAM) onto a single **XIAO ESP32-S3 Sense**. One module now handles camera capture, AI inference, motor control, and all peripherals, using dual-core affinity to keep motor timing isolated from bursty network and vision work.

The control system is **hierarchical**: a slow **planner** (~1 Hz, Core 1) calls Google's Gemini Robotics-ER to emit structured goals, and a fast **reactive executor** (~30 Hz, Core 0) drives the robot smoothly toward those goals while an ultrasonic sensor provides an independent obstacle reflex.

Core 0 — real-time

Reactive executor (visual servo, heading hold, motor PWM), peripheral I/O, command console, and the ultrasonic obstacle reflex.

Core 1 — bursty I/O

Planner (Gemini calls), OV2640 camera DMA, WiFi / MQTT / OTA.

What you get

A two-wheel-drive car that captures frames, asks Gemini what to do, and drives toward goals — with pan/tilt camera, status LEDs, an OLED display, buzzer feedback, WiFi provisioning over Bluetooth, and over-the-air updates.

2 · Bill of Materials

Qty	Component	Notes
1	XIAO ESP32-S3 Sense	MCU + OV2640 camera + 8 MB PSRAM, USB-C
1	TCA9548A I ² C multiplexer	Breakout, address 0x70
1	PCA9685 16-ch PWM driver	Breakout, address 0x40
1	TB6612FNG dual motor driver	Breakout
1	SSD1306 OLED display	128×64, I ² C, address 0x3C
1	Ultrasonic rangefinder	3.3 V variant : HC-SR04P / RCWL-1601 / US-100
2	RGB LED	Common-anode
2	SG90 micro servo	Pan / tilt
2	DC gear motor + wheel	~3–6 V hobby motors
1	Piezo buzzer	Passive
1	100 Ω resistor	In series with buzzer
2	18650 Li-ion cell + holder	Battery pack
1	XL6009 boost converter	Regulated to 5 V
—	Chassis, wiring, headers	2WD car chassis, jumper wires, standoffs

Tools required

Soldering iron + solder, wire strippers, small screwdriver set, multimeter (for verifying 5 V rail and continuity), a USB-C cable, and a computer with Docker (for the containerized firmware build).

XIAO Pin	GPIO	Function	Notes
D8–D10	GPIO7–9	<i>spare</i>	Future SPI / expansion

I²C runs at **400 kHz**.

4.2 · I²C topology (TCA9548A @ 0x70)

Select the channel on the multiplexer **before** addressing any downstream device.

Channel	Device	Address
ch0	PCA9685 PWM driver (motors, servos, LEDs)	0x40 @ 200 Hz
ch1	SSD1306 OLED display (128×64)	0x3C
ch2–7	<i>reserved — IMU / ToF / future sensors</i>	—

4.3 · PCA9685 channel map (0x40, 200 Hz)

All motor direction, motor PWM, servo, and LED outputs go through the PCA9685. Motor direction pins use PCA9685 “full-on” (4096) / “full-off” (0); PWM pins use the full 12-bit range (0–4095).

Ch	Signal	Ch	Signal
0	Left LED — R	8	Motor R — IN1 (dir)
1	Left LED — G	9	Motor R — IN2 (dir)
2	Left LED — B	10	Motor R — PWM
3	Right LED — R	11	Motor L — IN1 (dir)
4	Right LED — G	12	Motor L — IN2 (dir)
5	Right LED — B	13	Motor L — PWM
6	Pan servo (SG90)	14–15	<i>reserved</i>
7	Tilt servo (SG90)		

200 Hz is a compromise between servo timing (ideal 50 Hz) and motor PWM smoothness — it works well for SG90s and the TB6612FNG.

4.4 · Ultrasonic rangefinder

Signal	Pin	Voltage	Function
TRIG	GPIO3 (D2)	3.3 V	10 µs pulse triggers a measurement
ECHO	GPIO4 (D3)	3.3 V	Pulse width encodes distance (RMT RX)
VCC	3.3 V	3.3 V	3.3 V variant only
GND	any GND	—	Shared ground

The sensor samples at ~20 Hz. **Obstacle reflex:** if distance < 15 cm, the executor immediately stops and reverses, independent of planner goals.

5 · Power

Power the car from a 2×**18650 pack** through an **XL6009 boost converter set to 5 V**. Distribute that 5 V rail to the XIAO 5 V pin, the TB6612FNG (VM + VCC), the PCA9685 (V+ and VCC), and the servos.

Golden rule

Common ground everywhere. Every module — boost converter, XIAO, motor driver, PCA9685, servos, sensors — must share one

The 3.3 V logic for the OLED, ultrasonic sensor, and TCA9548A comes from the XIAO's 3V3 pin. Keep motor/servo current (high, noisy) on the 5 V rail and logic on 3V3.

GND. Missing grounds cause brown-outs, I²C lockups, and erratic motion.

Brown-out detection is disabled in firmware because motor inrush was tripping it; a stiff 5 V supply and thick power wires matter.

6 · Assembly Steps

1. **Mount the mechanics.** Fit the two gear motors and wheels to the chassis, add the caster/third wheel, and mount the battery holder low and centered.
2. **Wire power first.** Connect the 18650 pack to the XL6009 input, set its output to **5.0 V with a multimeter before connecting anything else**, then run the 5 V and shared GND rails.
3. **Place the XIAO** and bring out I²C (GPIO5/6), STBY (GPIO1), buzzer (GPIO2), and the ultrasonic pins (GPIO3/4).
4. **Wire the I²C chain:** XIAO SDA/SCL → TCA9548A → PCA9685 (ch0) and OLED (ch1). Pull-ups on the breakouts are usually sufficient.
5. **Wire the motor driver:** PCA9685 ch8–13 → TB6612FNG inputs; STBY → GPIO1; motor outputs → the two DC motors; VM/VCC → 5 V.
6. **Add servos** (PCA9685 ch6/7) and **RGB LEDs** (ch0–5, common-anode).
7. **Add the buzzer** on GPIO2 through the 100 Ω resistor, and the ultrasonic sensor on GPIO3/4 (3.3 V power).
8. **Double-check the 3.3 V vs 5 V rails** and confirm common ground with a multimeter continuity test before first power-up.

7 · Build & Flash the Firmware

Builds run inside a container — **no local ESP-IDF install is required**. The XIAO uses native USB-Serial-JTAG (USB VID 0x303a), which just auto-detects.

```
# From the repo root
just robocar-unified::build                # containerized ESP-IDF v5.4 build
PORT=/dev/cu.usbmodem* just robocar-unified::flash # flash over USB-C
just robocar-unified::monitor              # serial console
# or, in one step:
just robocar-unified::flash-monitor
```

Can't enter download mode?

Hold **BOOT**, tap **RESET**, then release **BOOT** to force the bootloader, and re-run the flash command.

The flasher writes three images to an 8 MB, OTA-capable layout:

Offset	Image	Partition
0x0	build/bootloader/bootloader.bin	bootloader
0x8000	build/partition_table/partition-table.bin	partition table
0x12000	build/robocar-unified.bin	ota_0 (app)

8 · First Boot & Provisioning

8.1 · WiFi over Bluetooth (Improv)

No WiFi credentials are compiled in. On first boot the device advertises an **Improv WiFi BLE service**. Use a browser-based Improv provisioner (Chrome on desktop or Android) to send your SSID and password; they are stored in NVS and reused on later boots.

For local development you can instead copy `main/credentials.h.example` to `main/credentials.h` (gitignored) and hard-code credentials.

8.2 · Discovery & AI backend

After connecting, the car is reachable at **robocar-unified.local** via mDNS. The planner uses **Gemini Robotics-ER 1.6** to emit goals — `drive()`, `track()`, `rotate()`, and `stop()`.

8.3 · Over-the-air updates

OTA is enabled with app rollback. The updater pulls releases from the `laurigates/mcu-tinkering-lab` GitHub repo; `version.txt` (currently **v0.1.2**) is the single source of truth and is managed by `release-please`.

9 · Functional Checkout

Work through these after first flash, watching the serial monitor:

✓	Check	Expected result
☐	Boot log	No PSRAM / boot-loop errors; tasks pin to cores 0 & 1
☐	I ² C scan	TCA9548A, PCA9685, and OLED all detected
☐	OLED	Status screen renders (128×64)
☐	LEDs	Both RGB LEDs cycle / show status colors
☐	Buzzer	Audible startup tone
☐	Servos	Pan/tilt center, then sweep within limits
☐	Motors	Both wheels drive forward and reverse; STBY HIGH
☐	Ultrasonic	Distance readings track a hand moving closer/away
☐	Reflex	Car stops/reverses when an obstacle is < 15 cm
☐	WiFi	Provisions via Improv; <code>robocar-unified.local</code> resolves

10 · Troubleshooting

Symptom	Likely cause & fix
Boot loop on power-up	Wrong PSRAM mode. The Sense uses octal PSRAM (<code>CONFIG_SPIRAM_MODE_OCT=y</code>); don't change it.
Random resets under motor load	Weak 5 V rail / missing common ground. Use thicker power wires and verify the XL6009 holds 5 V under load.

Symptom	Likely cause & fix
No I ² C devices found	Not selecting the TCA9548A channel first, or SDA/SCL swapped. Check GPIO5=SDA, GPIO6=SCL.
OLED and PCA9685 conflict	Both bypassing the mux. Route each through its own TCA9548A channel (ch1 / ch0).
Motors don't move	STBY (GPIO1) not HIGH, or VM not on 5 V. Confirm TB6612FNG power and enable line.
Servos jitter	Shared noisy rail. Keep servo power on 5 V with common ground; 200 Hz PWM is expected.
Board won't flash	Force download mode: hold BOOT, tap RESET, release BOOT.
Damaged ECHO / no distance	Used a 5 V HC-SR04. Replace with a 3.3 V module (HC-SR04P).